ESP32-S3 Embedded Rust Laboratory Manual

Programming with ESP-HAL 1.1.1 (no_std)

Edition 1 - Stage 1 of 2 (Parts I, III, labs 1-12, Project 1). Written 4 November 2026.

How to read the verification labels

This manual was written without access to an Xtensa toolchain, so **no example in it has been compiled or run on hardware**. Every lab carries one of these labels:

Label	Meaning
Checked (A)	Module paths, function names and signatures were read in the esp-hal 1.1.1 documentation or source. The code has not been compiled.
Pattern (B)	Some signatures come from esp-hal 1.1.0 or 1.0.0 source, or from examples. The compiler must confirm them. A note lists exactly what to check.
Planned (C)	Not written yet. Listed in the Stage 2 table with the stability findings already known.

First action for every reader: run `cargo build --bin lab01_gpio_output`. If a later lab fails, the compiler message is the authority for names; fix the lab and treat this manual as the starting point, not the last word.

How to read the verification labels 1
Part I - Environment and project configuration 4
1.1 ESP32-S3 architecture in one page 4
1.2 Embedded Rust and no_std 4
1.3 Install the toolchain (Linux Mint) 4
1.4 Generate a project 4
1.5 Project files explained 5
1.6 Anatomy of a firmware: entry point, panic handler, peripherals 6
1.7 GPIO map and conflicts with the OV2640 camera 7
Part II - Peripheral laboratories (1-12) 9
Lab 1 - GPIO digital output 9
Lab 2 - GPIO digital input and pull-up/pull-down 10
Lab 3 - CPU clock configuration 12
Lab 4 - ADC analog-to-digital conversion 13
Lab 5 - LEDC PWM and LED brightness control 15
Lab 6 - Servo motor control with PWM 17
Lab 7 - Hardware timers and periodic events 19
Lab 8 - GPIO interrupts and interrupt handlers 20
Lab 9 - UART serial communication 22
Lab 10 - I2C master communication 24
Lab 11 - LCD1602 I2C display 26
Lab 12 - SPI master communication 27
Labs 13-23 - Stage 2 (planned) 29
Part III - ESP-HAL 1.1.1 API reference (verified subset) 31
Initialisation and clocks 31
GPIO 31
Time and delay 32
Timers and watchdog 32
UART 32
I2C master 33
SPI master 33
ADC 34
LEDC 34
Part IV - Integrated projects 35
Project 1 - Basic monitoring system (ADC + PWM + LCD) 35
Project 2 - RFID access control (Stage 2 plan) 37

Project 3 - IoT monitor with Embassy (Stage 2 plan) 37
Appendix A - Project directory 37
Appendix B - Checklist per laboratory 37
Appendix C - Sources 38

Part I - Environment and project configuration

1.1 ESP32-S3 architecture in one page

- **CPU:** dual-core Xtensa LX7, up to 240 MHz. esp-hal runs your `main` on core 0.
- **Memory:** 512 KB internal SRAM, optional external PSRAM (quad or octal depending on module) and external flash. Modules with octal PSRAM (for example N16R8) reserve GPIO33-37.
- **Peripherals used in this manual:** GPIO matrix (any signal to nearly any pin), LEDC (PWM), SAR ADC1/ADC2, I2C0/I2C1, SPI2/SPI3, UART0-2, TIMG0/1 timers and watchdogs, SYSTIMER, PCNT, RMT, MCPWM, TWAI (CAN), USB Serial/JTAG, USB OTG, Wi-Fi and Bluetooth LE radio.
- **Camera interface:** the on-board OV2640 uses the dedicated LCD_CAM peripheral plus an I2C-like SCCB bus. Its pins are fixed by your wiring (section 1.7).

1.2 Embedded Rust and no_std

Firmware has no operating system. `#![no_std]` removes the standard library and keeps `core`; `#![no_main]` removes the normal `main` so esp-hal can provide the reset entry point. There is no heap unless you add an allocator (the `esp-alloc` crate, Stage 2). Panics need a handler: `esp_backtrace` provides one and prints a backtrace to the serial console.

1.3 Install the toolchain (Linux Mint)

```
# 1. Rust and build prerequisites
sudo apt update && sudo apt install -y build-essential curl git pkg-config libssl-dev
libudev-dev
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"

# 2. Xtensa toolchain installer, flasher and project generator
cargo install espup --locked
cargo install espflash --locked
cargo install esp-generate --locked
espup install --targets esp32s3

# 3. Load the toolchain into every new shell
echo '. $HOME/export-esp.sh' >> ~/.bashrc

# 4. Serial port access (log out and in afterwards)
sudo usermod -aG dialout $USER
```

`espup` installs the Xtensa-enabled Rust compiler under the toolchain name **esp**; that is why `rust-toolchain.toml` says `channel = "esp"`. The USB cable must plug into the board port that exposes the USB Serial/JTAG or the UART bridge, depending on your board.

1.4 Generate a project

```
esp-generate --chip esp32s3 esp32s3-lab
```

The generator asks for options (unstable HAL features, logging, VS Code settings). **Compare its output with the files in this manual and prefer the generator output** where they differ: the linker and runner flags in

particular come from my recollection of the 1.x template, not from a verified copy. The project in this manual uses esp-hal pinned with `=1.1.1`, because 1.2.x is already published and changes unstable APIs.

1.5 Project files explained

Cargo.toml

```
[package]
name = "esp32s3-lab"
version = "0.1.0"
edition = "2021"
rust-version = "1.88"

[lib]
name = "esp32s3_lab"
path = "src/lib.rs"

[dependencies]
esp-hal = { version = "=1.1.1", features = ["esp32s3", "unstable"] }
esp-bootloader-esp-idf = { version = "0.5.0", features = ["esp32s3"] }
esp-backtrace = { version = "0.19", features = ["esp32s3", "panic-handler", "println"] }
esp-println = { version = "0.17", features = ["esp32s3"] }
critical-section = "1"
nb = "1"

[profile.dev]
opt-level = "s"        # debug builds are too slow for timing-sensitive peripherals

[profile.release]
opt-level = "s"
lto = "fat"
codegen-units = 1
debug = 2
```

- `esp-hal = "=1.1.1"` pins the exact version. Feature `esp32s3` selects the chip. Feature **`unstable`** is required here because in 1.1.1 these modules are unstable: `timer`, `delay`, `ledc`, `analog` (ADC), `dma`, `rmt`, `pcnt`, `mcpwm`, `twai`, and GPIO interrupts (`Io`, `listen`, `#[handler]`). Stable and complete in 1.1.1: `gpio` (basic input/output), `i2c`, `spi` (blocking master), `uart`, `time`, `clock`.
- `esp-bootloader-esp-idf 0.5.0` provides the app descriptor the ESP-IDF bootloader expects. 0.5.0 depends on the same `esp-metadata-generated ^0.4` as esp-hal 1.1.1. The newer 0.6.0 does not match.
- `esp-backtrace` and `esp-println` versions (0.19 / 0.17) are the 1.1-era pair by release dates, but their dependency lists were not readable. Run `cargo tree -d` and fix any duplicate esp-* crates.
- `[profile.dev] opt-level = "s"`: an unoptimised debug build is too slow for timing-sensitive code such as the LCD driver.

.cargo/config.toml

```
[build]
target = "xtensa-esp32s3-none-elf"

[target.xtensa-esp32s3-none-elf]
runner = "espflash flash --monitor"

[env]
ESP_LOG = "info"
```

```
[unstable]
build-std = ["core"]
```

Sets the Xtensa target, the `runner` (so `cargo run` flashes and opens the monitor) and `build-std = ["core"]` because Xtensa has no pre-built standard library. `ESP_LOG` sets the log level used by esp-println logging.

rust-toolchain.toml

```
[toolchain]
channel = "esp"
```

build.rs

```
fn main() {
    // Linker script handling is provided by esp-hal 1.x itself; this file only
    // adds a friendlier hint when the linker fails. Regenerate with
    // `esp-generate --chip esp32s3` and compare if the linker reports errors.
    println!("cargo:rustc-link-arg=-Tlinkall.x");
}
```

A build script runs on the host before compilation. Here it only passes the linker script. If your generator output differs (for example it prints an error-hint wrapper), keep the generator version.

src/lib.rs and src/bin/*.rs

`src/lib.rs` holds shared code (the LCD driver). Each file in `src/bin/` is one independent firmware image, built with `cargo build --bin <name>` and flashed with `cargo run --bin <name>`.

1.6 Anatomy of a firmware: entry point, panic handler, peripherals

```
#![no_std]
#![no_main]

use esp_backtrace as _;                      // 1. links the panic handler
esp_bootloader_esp_idf::esp_app_desc!();     // 2. app descriptor for the bootloader

use esp_hal::{clock::CpuClock, main};

#[main]                                      // 3. entry point: must be fn() -> !
fn main() -> ! {
    let peripherals = esp_hal::init(
        esp_hal::Config::default().with_cpu_clock(CpuClock::max()),
    );                                       // 4. clocks + peripheral tokens
    loop {}
}
```

- `#[esp_hal::main]` (feature `rt`, on by default) marks the entry function. It must never return: its type is `fn() -> !`.
- `esp_hal::init(Config)` returns a `Peripherals` struct and, per the 1.1.1 source, disables the software and hardware watchdogs for you. `Config` has only `with_cpu_clock` in 1.1.1; `CpuClock` has `_80MHz`, `_160MHz`, `_240MHz` and `max()`.
- `esp_backtrace as _` is imported only for its side effect (the panic handler); `as _` silences the unused-import warning.

Ownership and borrowing of peripheral tokens

`Peripherals` contains one field per peripheral, for example `peripherals.GPIO2` or `peripherals.I2C0`. Each is a zero-sized token that exists exactly once. Passing the token **by value** to a driver (`Output::new(p.GPIO2, ...)`) moves it: the compiler then refuses any second use of GPIO2. This is how Rust turns "do not use one pin for two peripherals" into a compile-time error. Drivers carry a lifetime `'d`; a driver holding a moved token cannot outlive it. Destructure fields you need and leave the rest untouched.

Error handling

Constructors that can fail return `Result`, for example `Uart::new(...) -> Result<Uart, ConfigError>` and `I2c::new(...) -> Result<I2c, ConfigError>`. The labs call `.unwrap()` for brevity; production code should match the error. Bus operations return `Result<_, i2c::master::Error>` or `spi::Error`. ADC reads return `nb::Result<u16, ()>`, where `nb::Error::WouldBlock` means "not ready yet".

1.7 GPIO map and conflicts with the OV2640 camera

Your camera and display use these pins. Everything else must avoid them while the camera is active:

Function	GPIO	Notes
Camera XCLK	15	LCD_CAM clock output
Camera SIOD / SIOC	4 / 5	SCCB (I2C-like) bus to the sensor
Camera D0-D7	11, 9, 8, 10, 12, 18, 17, 16	Parallel pixel data
Camera VSYNC / HREF / PCLK	6 / 7 / 13	Frame sync, line valid, pixel clock
LCD1602 SDA / SCL	41 / 40	I2C0, address 0x27

Used by the camera: GPIO4-13 and GPIO15-18. These are the pins that would otherwise be convenient for ADC, PWM and SPI.

Consequences

- **ADC1 is GPIO1-10 and ADC2 is GPIO11-20** (ESP32-S3 datasheet; verify for your module). With the camera attached, the only free ADC1 pins are **GPIO1, GPIO2 and GPIO3**. GPIO14 is free on ADC2. ADC2 conflicts with Wi-Fi in ESP-IDF; prefer ADC1 pins whenever Wi-Fi is enabled.
- **Unavailable regardless of the camera:** GPIO26-32 (flash and quad PSRAM), GPIO33-37 on octal-PSRAM modules, GPIO19/20 (USB D-/D+ when you use USB), and the strapping pins GPIO0, GPIO3, GPIO45, GPIO46, which should not be pulled to a level that changes boot mode. GPIO43/44 are the default UART0 console on most boards.
- **The RGB LED** on DevKitC-1 boards sits on GPIO48 (v1.0) or GPIO38 (v1.1); check your board before driving it.
- **The camera SCCB bus (GPIO4/5) and the LCD bus (GPIO40/41)** are separate, so there is no conflict between them. Never put the LCD on GPIO4/5 while the camera is attached, and never configure GPIO4-13/15-18 in a lab while the camera is physically connected and active.

Pins used in this manual

Signal	GPIO	Used in
LED output / LED PWM	2	Labs 1, 2, 5, 7; Project 1
ADC sensor input	1	Lab 4; Project 1
Push button to GND	21	Labs 2, 8
Servo signal	14	Lab 6; Project 2
LCD SDA / SCL	41 / 40	Labs 10, 11; Projects 1-3
SPI SCK / MOSI / MISO	47 / 48 / 42	Lab 12; Project 2 (RC522)
RC522 CS / RST (planned)	38 / 39	Project 2
UART1 TX / RX loopback	47 / 48	Lab 9 only (same pins as SPI lab: never run both at once)

Within one firmware image, no pin is assigned twice. Labs 9 and 12 reuse GPIO47/48 because they are separate images; do not wire both loopback jumpers at the same time.

Part II - Peripheral laboratories (1-12)

Build and flash any lab with: `cargo run --release --bin <name>` (the runner flashes with espflash and opens the serial monitor). Exit the monitor with Ctrl+R then Ctrl+C or Ctrl+C depending on the espflash version.

Lab 1 - GPIO digital output

Status: signatures checked against esp-hal 1.1.1 documentation/source. NOT compiled or run on hardware.

Objective

Drive an LED from a GPIO pin and print its state.

Required components

ESP32-S3 board, LED, 330 ohm resistor, breadboard, jumper wires.

Wiring

From	Through	To
GPIO2	Resistor 330 ohm	LED anode
LED cathode		GND

Hardware principle

A GPIO output drives the pin to 0 V or 3.3 V through a push-pull stage. The resistor limits current: (3.3 V - 2 V) / 330 ohm is about 4 mA. The pin never sources more than the configured drive strength (default 20 mA).

Source code

File: `src/bin/lab01_gpio_output.rs`

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use esp_hal::{
    clock::CpuClock,
    delay::Delay,
    gpio::{Level, Output, OutputConfig},
    main,
};

#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default().with_cpu_clock(CpuClock::max()));

    // GPIO2 is moved into the driver: nobody else can use it now.
    let mut led = Output::new(p.GPIO2, Level::Low, OutputConfig::default());
    let delay = Delay::new();
```

```
    loop {
        led.toggle();
        println!("LED is {}", if led.is_set_high() { "ON" } else { "OFF" });
        delay.delay_millis(500);
    }
}
```

Explanation

- `Output::new(pin, initial_level, config)`: moves the pin token into the driver and sets its initial level before enabling the output, so there is no glitch.
- `OutputConfig::default()`: push-pull, 20 mA drive strength, no pull. Builders `with_drive_mode`, `with_drive_strength`, `with_pull` exist.
- `toggle()`, `set_high()`, `set_low()`, `set_level(Level)`: all take `&mut self`, so `led` must be `let mut`.
- `Delay::new()` and `delay_millis(u32)`: `esp_hal::delay` is an **unstable** module in 1.1.1; the `unstable` feature is enabled in Cargo.toml.

Expected behavior

The LED blinks at 1 Hz; the console prints "LED is ON" / "LED is OFF" alternately.

Build and flash

```
cargo run --release --bin lab01_gpio_output
```

Common errors and troubleshooting

- Compile error "module `delay` is private / unstable": enable feature `unstable` on esp-hal.
- Nothing printed: open the monitor on the correct serial port; if the board has two USB ports, use the one wired to the USB/UART bridge.
- LED dark: check polarity and that GPIO2 is not also driving another board LED.

Documentation

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-RS book: https://esp-rs.github.io/book/

Lab 2 - GPIO digital input and pull-up/pull-down

Status: signatures checked against esp-hal 1.1.1 documentation/source. NOT compiled or run on hardware.

Objective

Read a push button with the internal pull-up and mirror it on an LED.

Required components

Board, push button, LED + 330 ohm (from lab 1), jumper wires.

Wiring

From	Through	To
GPIO21	Button terminal 1	

From	Through	To
Button terminal 2		GND
GPIO2	330 ohm to LED	as lab 1

Hardware principle

A floating input reads noise. A pull-up resistor (about 45 kohm internal) holds the pin high; pressing the button connects it to GND, so the pin reads low. With `Pull::Down` the logic is inverted and the button must go to 3.3 V.

Source code

File: `src/bin/lab02_gpio_input.rs`

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use esp_hal::{
    delay::Delay,
    gpio::{Input, InputConfig, Level, Output, OutputConfig, Pull},
    main,
};

#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default());

    let mut led = Output::new(p.GPIO2, Level::Low, OutputConfig::default());
    // Button between GPIO21 and GND; internal pull-up keeps the pin HIGH when released.
    let button = Input::new(p.GPIO21, InputConfig::default().with_pull(Pull::Up));
    let delay = Delay::new();

    loop {
        let pressed = button.is_low();
        led.set_level(Level::from(pressed));
        delay.delay_millis(10);
    }
}
```

Explanation

- `Input::new(pin, InputConfig)`: pin becomes an input.
- `InputConfig::default().with_pull(Pull::Up)`: `Pull` has `None`, `Up`, `Down`.
- `is_low()` / `is_high()`: read the level without blocking.
- `Level::from(bool)`: converts the boolean to `Level` (the 1.1.1 docs list `From<bool>`).

Expected behavior

The LED is on only while the button is pressed.

Build and flash

```
cargo run --release --bin lab02_gpio_input
```

Common errors and troubleshooting

- LED always on: button wired to 3.3 V instead of GND, or `Pull::Down` used.
- Erratic: the internal pull-up is weak; for long wires add an external 10 kohm resistor.

Documentation

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-RS book: https://esp-rs.github.io/book/

Lab 3 - CPU clock configuration

Status: signatures checked against esp-hal 1.1.1 documentation/source. NOT compiled or run on hardware.

Objective

Change the CPU frequency and measure its effect on a software loop.

Required components

Board and USB cable only.

Wiring

No external wiring.

Hardware principle

The CPU clock is derived from the PLL. At 240 MHz a simple loop executes about three times as many iterations per second as at 80 MHz. Higher clocks raise current consumption; peripherals on the APB bus stay at a fixed 80 MHz.

Source code

File: `src/bin/lab03_cpu_clock.rs`

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use esp_hal::{
    clock::CpuClock,
    main,
    time::{Duration, Instant},
};

// Rebuild with CpuClock::_80MHz, _160MHz and _240MHz and compare the printed counts.
const CLOCK: CpuClock = CpuClock::_240MHz;

#[main]
fn main() -> ! {
```

```
    let _p = esp_hal::init(esp_hal::Config::default().with_cpu_clock(CLOCK));

    loop {
        let start = Instant::now();
        let mut n: u32 = 0;
        while start.elapsed() < Duration::from_secs(1) {
            n = n.wrapping_add(1);
            core::hint::black_box(n);
        }
        println!("loop iterations in 1 s: {}", n);
    }
}
```

Explanation

- `esp_hal::Config::default().with_cpu_clock(CpuClock::_80MHz)`: the only field of `Config` in 1.1.1.
- `CpuClock::{_80MHz, _160MHz, _240MHz}` and `CpuClock::max()`.
- `Instant::now()` and `.elapsed()`: `esp_hal::time`, a 1 microsecond free-running counter.
- `core::hint::black_box` stops the optimiser from deleting the loop.

Expected behavior

Printed counts scale roughly with the CPU frequency (for example 80 MHz gives about one third of the 240 MHz count).

Build and flash

```
cargo run --release --bin lab03_cpu_clock
```

Common errors and troubleshooting

- Identical counts at every setting: the loop was optimised away or the same binary was flashed again; rebuild after editing `CLOCK`.

Documentation

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-RS book: https://esp-rs.github.io/book/

Lab 4 - ADC analog-to-digital conversion

Status: signatures checked against esp-hal 1.1.1 documentation/source. NOT compiled or run on hardware.

Objective

Read a potentiometer or LDR divider with the SAR ADC.

Required components

Board, 10 kohm potentiometer.

Wiring

From	Through	To
Potentiometer outer pin 1		3.3 V

From	Through	To
Potentiometer outer pin 2		GND
Potentiometer wiper	GPIO1	(ADC1 channel 0)

Hardware principle

The ADC compares the input voltage with a reference using successive approximation and returns a 12-bit code (0-4095). Attenuation scales the input range: `_0dB` about 0-1.1 V up to `_11dB` about 0-3.1 V. The ADC is not linear near the ends; calibration support exists (`enable_pin_with_cal`) but is not used here.

Source code

File: `src/bin/lab04_adc.rs`

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use esp_hal::{
    analog::adc::{Adc, AdcConfig, Attenuation},
    delay::Delay,
    main,
};

#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default());

    // GPIO1 = ADC1 channel 0 on the ESP32-S3.
    let mut cfg = AdcConfig::new();
    let mut pin = cfg.enable_pin(p.GPIO1, Attenuation::_11dB);
    let mut adc1 = Adc::new(p.ADC1, cfg);
    let delay = Delay::new();

    loop {
        // read_oneshot returns nb::Result<u16, ()>; nb::block! retries on WouldBlock.
        let raw: u16 = nb::block!(adc1.read_oneshot(&mut pin)).unwrap();
        println!("ADC raw = {:4}  (~{} %)", raw, (raw as u32) * 100 / 4095);
        delay.delay_millis(200);
    }
}
```

Explanation

- `AdcConfig::new()` and `enable_pin(pin, Attenuation)` return an `AdcPin`; the pin must be an ADC-capable one.
- `Adc::new(peripherals.ADC1, config)`: on the S3 `ADC1` serves GPIO1-10.
- `read_oneshot(&mut pin) -> nb::Result<u16, ()>` and `nb::block!` to wait until the result is ready.
- The `analog` module is **unstable** in 1.1.1.

Expected behavior

Values between about 0 and 4095 that follow the knob; percent is printed alongside.

Build and flash

```
cargo run --release --bin lab04_adc
```

Common errors and troubleshooting

- Always 4095 or 0: wiper wired to the wrong pin, or attenuation too low for the voltage.
- Compile error about `nb`: `nb = "1"` must be in Cargo.toml.
- With the camera attached do not use GPIO4-10; GPIO1-3 remain free.

Documentation

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-RS book: https://esp-rs.github.io/book/

Lab 5 - LEDC PWM and LED brightness control

Status: partly checked against esp-hal 1.1.1, partly against 1.1.0 / 1.0.0 source (see notes). NOT compiled. Treat as an implementation pattern and let the compiler confirm names.

Objective

Fade an LED with hardware PWM.

Required components

Board, LED, 330 ohm resistor.

Wiring

From	Through	To
GPIO2	330 ohm	LED anode, cathode to GND

Hardware principle

PWM switches the output at a fixed frequency; the duty cycle (fraction of time high) sets the average power. LEDC has timers (frequency and resolution) and channels (one pin each). At 5 kHz with 8-bit resolution the divider is 80 MHz / (5000 x 256) = 62.5, which is within the legal range.

Source code

File: `src/bin/lab05_ledc_pwm.rs`

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use esp_hal::{
```

```
    delay::Delay,
    gpio::DriveMode,
    ledc::{
        channel::{self, ChannelIFace},
        timer::{self, TimerIFace},
        LSGlobalClkSource, Ledc, LowSpeed,
    },
    main,
    time::Rate,
};

#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default());

    let mut ledc = Ledc::new(p.LEDC);
    ledc.set_global_slow_clock(LSGlobalClkSource::APBClk);

    let mut timer0 = ledc.timer::<LowSpeed>(timer::Number::Timer0);
    timer0
        .configure(timer::config::Config {
            duty: timer::config::Duty::Duty8Bit,
            clock_source: timer::LSClockSource::APBClk,
            frequency: Rate::from_khz(5),
        })
        .unwrap();

    let mut ch0 = ledc.channel::<LowSpeed>(channel::Number::Channel0, p.GPIO2);
    ch0.configure(channel::config::Config {
        timer: &timer0,
        duty_pct: 0,
        drive_mode: DriveMode::PushPull,
    })
    .unwrap();

    let delay = Delay::new();
    loop {
        for pct in (0..=100u8).chain((0..100u8).rev()) {
            ch0.set_duty(pct).unwrap();
            delay.delay_millis(10);
        }
    }
}
```

Explanation

- `Ledc::new(peripherals.LEDC)` and `set_global_slow_clock(LSGlobalClkSource::APBClk)`.
- `ledc.timer::<LowSpeed>(timer::Number::Timer0)` then `configure(timer::config::Config { duty, clock_source, frequency })`. On the S3 only low-speed mode exists.
- `ledc.channel::<LowSpeed>(channel::Number::Channel0, pin)` then `configure(channel::config::Config { timer, duty_pct, drive_mode })`.
- `set_duty(pct: u8)` takes whole percent.
- Traits `TimerIFace` and `ChannelIFace` must be imported to call `configure`/`set_duty`.

Expected behavior

Smooth fade in and out about every two seconds.

Build and flash

```
cargo run --release --bin lab05_ledc_pwm
```

Common errors and troubleshooting

- `Error::Divisor`: frequency x 2^bits is too large for the clock; lower the resolution or frequency.
- **Check with the compiler:** the `drive_mode` field and its type were read in the 1.1.0 source; if 1.1.1 renamed it, follow the compile error.

Documentation

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-RS book: https://esp-rs.github.io/book/

Lab 6 - Servo motor control with PWM

Status: partly checked against esp-hal 1.1.1, partly against 1.1.0 / 1.0.0 source (see notes). NOT compiled. Treat as an implementation pattern and let the compiler confirm names.

Objective

Sweep a hobby servo between 1 ms and 2 ms pulses at 50 Hz.

Required components

Board, SG90 or MG90 servo, external 5 V supply for the servo (recommended), jumper wires.

Wiring

From	Through	To
GPIO14	Servo signal (orange/yellow)	
Servo V+ (red)		5 V supply
Servo GND (brown)		GND shared with the board

Hardware principle

A hobby servo expects a pulse every 20 ms whose width (about 1-2 ms) sets the angle. At 50 Hz, 1 ms is 5 % duty and 2 ms is 10 %. With 80 MHz APB the timer needs at least 11-bit resolution to reach 50 Hz (10-bit gives an invalid divider); 14-bit is used.

Source code

File: `src/bin/lab06_servo.rs`

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use esp_hal::{
    delay::Delay,
```

```
    gpio::DriveMode,
    ledc::{
        channel::{self, ChannelIFace},
        timer::{self, TimerIFace},
        LSGlobalClkSource, Ledc, LowSpeed,
    },
    main,
    time::Rate,
};

// 50 Hz => 20 ms period. 1 ms pulse = 5 %, 1.5 ms = 7.5 %, 2 ms = 10 %.
// With an 80 MHz APB clock, 14-bit resolution is valid at 50 Hz (10-bit is not).
#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default());

    let mut ledc = Ledc::new(p.LEDC);
    ledc.set_global_slow_clock(LSGlobalClkSource::APBClk);

    let mut timer0 = ledc.timer::<LowSpeed>(timer::Number::Timer0);
    timer0
        .configure(timer::config::Config {
            duty: timer::config::Duty::Duty14Bit,
            clock_source: timer::LSClockSource::APBClk,
            frequency: Rate::from_hz(50),
        })
        .unwrap();

    let mut servo = ledc.channel::<LowSpeed>(channel::Number::Channel0, p.GPIO14);
    servo
        .configure(channel::config::Config {
            timer: &timer0,
            duty_pct: 5,
            drive_mode: DriveMode::PushPull,
        })
        .unwrap();

    let delay = Delay::new();
    loop {
        // set_duty takes whole percent: only 6 positions between 5 % and 10 %.
        for pct in 5..=10u8 {
            servo.set_duty(pct).unwrap();
            println!("servo duty {} %", pct);
            delay.delay_millis(600);
        }
        for pct in (5..10u8).rev() {
            servo.set_duty(pct).unwrap();
            delay.delay_millis(600);
        }
    }
}
```

Explanation

- `Duty14Bit` resolution with `Rate::from_hz(50)`.
- `set_duty(5..=10)`: only six positions because `set_duty` takes whole percent. For finer control a raw hardware duty function exists in the 1.1.0 source (`set_duty_hw`) but its accessibility in 1.1.1 was not confirmed.
- Share GND between the servo supply and the board.

Expected behavior

The horn steps through six positions and back, with a print per step.

Build and flash

```
cargo run --release --bin lab06_servo
```

Common errors and troubleshooting

- Servo jitters or board resets: powering the servo from the board 5 V pin; use a separate supply.
- No movement: the signal wire is on a pin other than GPIO14, or GND is not shared.

Documentation

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-RS book: https://esp-rs.github.io/book/

Lab 7 - Hardware timers and periodic events

Status: partly checked against esp-hal 1.1.1, partly against 1.1.0 / 1.0.0 source (see notes). NOT compiled. Treat as an implementation pattern and let the compiler confirm names.

Objective

Toggle an LED from a hardware periodic timer.

Required components

Board, LED + resistor on GPIO2.

Wiring

From	Through	To
GPIO2	330 ohm	LED anode

Hardware principle

The timer group peripheral counts the 80 MHz APB clock (via a prescaler) and raises an event every period. This is more precise than software delay loops and lets the CPU do other work between events.

Source code

File: `src/bin/lab07_timer.rs`

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use esp_hal::{
    gpio::{Level, Output, OutputConfig},
    main,
    time::Duration,
```

```
    timer::{timg::TimerGroup, PeriodicTimer},
};

#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default());

    let mut led = Output::new(p.GPIO2, Level::Low, OutputConfig::default());

    let tg0 = TimerGroup::new(p.TIMG0);
    let mut timer = PeriodicTimer::new(tg0.timer0);
    timer.start(Duration::from_millis(500)).unwrap();

    loop {
        timer.wait(); // blocks until the period elapsed
        led.toggle();
        println!("tick");
    }
}
```

Explanation

- `TimerGroup::new(peripherals.TIMG0)` exposes `timer0`, `timer1` and `wdt`.
- `PeriodicTimer::new(timer)`, `start(Duration)`, `wait()`.
- Interrupt-driven use: `listen`, `set_interrupt_handler`, `clear_interrupt` (all in the unstable `timer` module); not used here.
- `SystemTimer::new(peripherals.SYSTIMER).alarm0` can also be passed to `PeriodicTimer::new` (not tested).

Expected behavior

The LED toggles every 500 ms and "tick" is printed.

Build and flash

```
cargo run --release --bin lab07_timer
```

Common errors and troubleshooting

- **Check with the compiler:** whether `wait()` returns `()` or `nb::Result<(), _>` in your exact build; if it returns `nb::Result`, wrap it with `nb::block!`.

Documentation

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-RS book: https://esp-rs.github.io/book/

Lab 8 - GPIO interrupts and interrupt handlers

Status: partly checked against esp-hal 1.1.1, partly against 1.1.0 / 1.0.0 source (see notes). NOT compiled. Treat as an implementation pattern and let the compiler confirm names.

Objective

Count button presses in an interrupt handler.

Required components

Board, push button.

Wiring

From	Through	To
GPIO21	Button	GND

Hardware principle

An edge on the pin sets a bit in the interrupt status register; the CPU jumps to the handler. The handler must be short, must clear the status (esp-hal does not do it for you) and may only touch data protected against concurrent access.

Source code

File: `src/bin/lab08_gpio_interrupt.rs`

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use core::{
    cell::RefCell,
    sync::atomic::{AtomicU32, Ordering},
};
use critical_section::Mutex;
use esp_hal::{
    delay::Delay,
    gpio::{Event, Input, InputConfig, Io, Pull},
    handler, main,
};

static BUTTON: Mutex<RefCell<Option<Input<'static>>>> = Mutex::new(RefCell::new(None));
static PRESSES: AtomicU32 = AtomicU32::new(0);

#[handler]
fn gpio_isr() {
    critical_section::with(|cs| {
        if let Some(btn) = BUTTON.borrow_ref_mut(cs).as_mut() {
            if btn.is_interrupt_set() {
                PRESSES.fetch_add(1, Ordering::Relaxed);
                btn.clear_interrupt(); // the driver does NOT clear the status for you
            }
        }
    });
}

#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default());

    let mut io = Io::new(p.IO_MUX);
    io.set_interrupt_handler(gpio_isr);
```

```
    let mut button = Input::new(p.GPIO21, InputConfig::default().with_pull(Pull::Up));
    critical_section::with(|cs| {
        button.listen(Event::FallingEdge);
        BUTTON.borrow_ref_mut(cs).replace(button);
    });

    let delay = Delay::new();
    let mut last = 0;
    loop {
        let n = PRESSES.load(Ordering::Relaxed);
        if n != last {
            println!("presses (incl. bounces): {}", n);
            last = n;
        }
        delay.delay_millis(50);
    }
}
```

Explanation

- `Io::new(peripherals.IO_MUX)` and `set_interrupt_handler(handler)` install one shared GPIO handler.
- `#[esp_hal::handler]` marks the interrupt function.
- `input.listen(Event::FallingEdge);` `Event` has `RisingEdge`, `FallingEdge`, `AnyEdge`, `LowLevel`, `HighLevel`.
- `is_interrupt_set()` and `clear_interrupt()` inside the handler. Level events fire continuously unless you call `unlisten()`.
- Sharing: `Mutex<RefCell<Option<Input<'static>>>>` from `critical-section` gives the handler access to the pin; `AtomicU32` carries the count to `main`.

Expected behavior

Each press increments the counter; mechanical bounce usually produces several counts per press (a debounce step is an exercise).

Build and flash

```
cargo run --release --bin lab08_gpio_interrupt
```

Common errors and troubleshooting

- The handler fires endlessly: the status was not cleared, or a level event is in use.
- **Pattern note:** the sharing pattern is reconstructed from the documentation, not copied from an official example; compile and test it.

Documentation

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-RS book: https://esp-rs.github.io/book/

Lab 9 - UART serial communication

Status: signatures checked against esp-hal 1.1.1 documentation/source. NOT compiled or run on hardware.

Objective

Send bytes through UART1 and read them back with a loopback jumper.

Required components

Board, one jumper wire.

Wiring

From	Through	To
GPIO47 (TX)	Jumper	GPIO48 (RX)

Hardware principle

UART sends bytes asynchronously as start bit, 8 data bits, optional parity, stop bit(s) at an agreed baud rate. The controller buffers bytes in a 128-byte FIFO. Do not use UART0 here: on most boards it carries the console.

Source code

File: `src/bin/lab09_uart.rs`

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use esp_hal::{
    delay::Delay,
    main,
    uart::{Config, Uart},
};

// Loopback test: wire GPIO47 (TX) to GPIO48 (RX) with a jumper.
#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default());

    let mut uart = Uart::new(p.UART1, Config::default().with_baudrate(115_200))
        .unwrap()
        .with_tx(p.GPIO47)
        .with_rx(p.GPIO48);
    let delay = Delay::new();

    loop {
        let msg = b"hello";
        let _ = uart.write(msg).unwrap();
        uart.flush().unwrap();

        let mut buf = [0u8; 5];
        let n = uart.read(&mut buf).unwrap(); // blocks until at least one byte arrives
        println!("received {} byte(s): {:?}", n, &buf[..n]);
        delay.delay_millis(1000);
    }
}
```

Explanation

- `Uart::new(peripherals.UART1, Config::default().with_baudrate(115_200))` returns `Result`.
- `.with_tx(pin)` and `.with_rx(pin)` connect the signals to GPIO through the matrix.
- `write(&[u8]) -> Result<usize, TxError>` may write fewer bytes than requested; `flush()` waits for the FIFO to drain.
- `read(&mut [u8]) -> Result<usize, RxError>` blocks until at least one byte is available.
- Unstable in 1.1.1: `split`, interrupts, `read_buffered`, flow control, break detection.

Expected behavior

"received 5 byte(s): [104, 101, 108, 108, 111]" every second.

Build and flash

```
cargo run --release --bin lab09_uart
```

Common errors and troubleshooting

- Hangs in `read`: the jumper is missing.
- `RxError::FrameFormatViolated`: baud mismatch or floating RX line.

Documentation

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-RS book: https://esp-rs.github.io/book/

Lab 10 - I2C master communication

Status: signatures checked against esp-hal 1.1.1 documentation/source. NOT compiled or run on hardware.

Objective

Scan the bus and find the LCD backpack (expected 0x27, sometimes 0x3F).

Required components

Board, LCD1602 with PCF8574 I2C backpack.

Wiring

From	Through	To
GPIO41	LCD SDA	
GPIO40	LCD SCL	
LCD VCC		see warning below
LCD GND		GND

Hardware principle

I2C uses two open-drain lines (SDA, SCL) with pull-up resistors. The master sends START, a 7-bit address plus R/W bit, and the addressed slave pulls SDA low to ACK. Addresses are given **without** the R/W bit.

Source code

File: `src/bin/lab10_i2c_scan.rs`

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use esp_hal::{
    delay::Delay,
    i2c::master::{Config, I2c},
    main,
};

#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default());

    // LCD backpack bus: SDA = GPIO41, SCL = GPIO40.
    let mut i2c = I2c::new(p.I2C0, Config::default())
        .unwrap()
        .with_sda(p.GPIO41)
        .with_scl(p.GPIO40);
    let delay = Delay::new();

    loop {
        println!("scanning...");
        for addr in 0x08u8..=0x77 {
            // An empty write sends only START + address; an ACK means a device is
present.
            if i2c.write(addr, &[]).is_ok() {
                println!("  device at 0x{:02X}", addr);
            }
        }
        delay.delay_millis(3000);
    }
}
```

Explanation

- `I2c::new(peripherals.I2C0, Config::default())` returns `Result`; default speed is 100 kHz.
- `.with_sda(pin)`, `.with_scl(pin)` set the pin to open-drain with the **internal** pull-up enabled.
- `write(addr, &[])`: an empty write transmits only the address; `Ok` means ACK. Zero-length *reads* are rejected with `Error::ZeroLengthInvalid`.
- A plain `u8` converts to a 7-bit `I2cAddress`.

Expected behavior

"device at 0x27" printed every three seconds.

Build and flash

```
cargo run --release --bin lab10_i2c_scan
```

Common errors and troubleshooting

- Nothing found: swap SDA/SCL, check solder on the backpack, try address 0x3F.
- **Voltage warning (important):** the ESP32-S3 GPIO is 3.3 V and not 5 V tolerant. Most LCD backpacks pull SDA/SCL up to their VCC. If the backpack is powered at 5 V with its pull-ups, 5 V reaches the ESP32 pins. Remove the two pull-up resistors (or solder jumpers) on the backpack and use 4.7 kohm pull-ups to 3.3 V, or use a bidirectional level shifter. The display itself usually needs 5 V for readable contrast.

Documentation

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-RS book: https://esp-rs.github.io/book/

Lab 11 - LCD1602 I2C display

Status: partly checked against esp-hal 1.1.1, partly against 1.1.0 / 1.0.0 source (see notes). NOT compiled. Treat as an implementation pattern and let the compiler confirm names.

Objective

Write text and a live counter to a 16x2 character display.

Required components

Board, LCD1602 + PCF8574 backpack (address 0x27); see the voltage warning in lab 10.

Wiring

From	Through	To
GPIO41	LCD SDA	
GPIO40	LCD SCL	

Hardware principle

The backpack is an 8-bit I/O expander wired to the HD44780 controller in 4-bit mode: P0=RS, P1=RW, P2=EN, P3=backlight, P4-P7=D4-D7. Each byte is sent as two nibbles, each latched by a pulse on EN. Initialisation sends 0x3 three times, then 0x2 to enter 4-bit mode, then the function-set (0x28), display-on (0x0C), entry-mode (0x06) and clear (0x01) commands.

Source code

File: `src/bin/lab11_lcd1602.rs`

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use core::fmt::Write;
use esp32s3_lab::lcd1602::Lcd;
use esp_hal::{
    delay::Delay,
    i2c::master::{Config, I2c},
```

```
    main,
};

#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default());

    let i2c = I2c::new(p.I2C0, Config::default())
        .unwrap()
        .with_sda(p.GPIO41)
        .with_scl(p.GPIO40);

    let mut lcd = Lcd::new(i2c, 0x27);
    lcd.init();
    lcd.print("ESP32-S3 Rust");

    let delay = Delay::new();
    let mut n: u32 = 0;
    loop {
        lcd.set_cursor(0, 1);
        write!(lcd, "count = {:6}", n).ok();
        n += 1;
        delay.delay_millis(500);
    }
}
```

Explanation

- `src/lcd1602.rs` implements the driver on top of `I2c<Blocking>`; the pin mapping is the common one, and some backpacks differ.
- `core::fmt::Write` is implemented for `Lcd`, so `write!(lcd, "count = {:6}", n)` works.
- The I2C driver is moved into `Lcd` (ownership transfer).

Expected behavior

Row 1 shows "ESP32-S3 Rust"; row 2 shows a counter.

Build and flash

```
cargo run --release --bin lab11_lcd1602
```

Common errors and troubleshooting

- Backlight but no characters: adjust the contrast potentiometer on the backpack.
- Garbage characters: your backpack uses a different bit mapping; change `RS`, `EN`, `BL` constants and the nibble shift.

Documentation

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-RS book: https://esp-rs.github.io/book/

Lab 12 - SPI master communication

Status: signatures checked against esp-hal 1.1.1 documentation/source. NOT compiled or run on hardware.

Objective

Run an SPI transfer and check it with a MOSI-MISO loopback.

Required components

Board, one jumper wire.

Wiring

From	Through	To
GPIO47		SCK (unused by loopback)
GPIO48 (MOSI)	Jumper	GPIO42 (MISO)

Hardware principle

SPI is a synchronous full-duplex bus: the master clocks bits out on MOSI while clocking bits in on MISO. Mode 0 means clock idles low and data is sampled on the rising edge. A hardware chip select is optional; the RC522 project uses a normal GPIO.

Source code

File: `src/bin/lab12_spi_loopback.rs`

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use esp_hal::{
    delay::Delay,
    main,
    spi::{
        master::{Config, Spi},
        Mode,
    },
    time::Rate,
};

// Loopback test: wire GPIO48 (MOSI) to GPIO42 (MISO) with a jumper.
#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default());

    let mut spi = Spi::new(
        p.SPI2,
        Config::default()
            .with_frequency(Rate::from_khz(100))
            .with_mode(Mode::_0),
    )
    .unwrap()
    .with_sck(p.GPIO47)
    .with_mosi(p.GPIO48)
    .with_miso(p.GPIO42);
    let delay = Delay::new();

    loop {
```

```
        let mut buf = [0xDE, 0xAD, 0xBE, 0xEF];
        spi.transfer(&mut buf).unwrap(); // full duplex: tx bytes are replaced by rx
bytes
        println!("loopback returned {:02X?}", buf);
        delay.delay_millis(1000);
    }
}
```

Explanation

- `Spi::new(peripherals.SPI2, Config::default().with_frequency(Rate::from_khz(100)).with_mode(Mode::_0))` returns `Result`; frequencies below 70 kHz are rejected with `ConfigError::FrequencyOutOfRange`.
- `.with_sck`, `.with_mosi`, `.with_miso` assign pins.
- `transfer(&mut [u8])` sends the buffer and overwrites it with the received bytes.
- Unstable in 1.1.1: `with_cs` (one hardware CS line only), DMA (`with_dma`, `with_buffers`), half-duplex, quad/octal modes.
- The driver also implements the embedded-hal `SpiBus` trait, which most device crates expect.

Expected behavior

"loopback returned [DE, AD, BE, EF]". Without the jumper the buffer returns zeros or 0xFF.

Build and flash

```
cargo run --release --bin lab12_spi_loopback
```

Common errors and troubleshooting

- Always zeros: MOSI-MISO jumper missing.
- `FrequencyOutOfRange`: use 70 kHz to 80 MHz.

Documentation

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-RS book: https://esp-rs.github.io/book/

Labs 13-23 - Stage 2 (planned)

These labs are not written yet. What is already known from the 1.1.1 documentation is listed so you can plan around it:

Lab	Stability / notes found so far
13 RC522 over SPI	Needs a device driver or hand-written register access on top of lab 12 (stable blocking SPI + GPIO CS and RST).
14 DMA	Module `dma` and `Spi::with_dma`/`with_buffers` are unstable. ESP32-S3 uses PDMA channels; PSRAM buffers need 32-byte alignment. Exact buffer constructors not yet verified in 1.1.1.
15 Watchdog	`TimerGroup::new(TIMG0).wdt` with `enable`, `feed`, `set_timeout(MwdtStage, Duration)`, `disable`; `esp_hal::init` disables watchdogs by default. Lives in the unstable `timer` module.

Lab	Stability / notes found so far
16 RMT	Unstable. 1.1.1 uses a new transaction-style API (`Rmt::new`, `configure_tx`/`configure_rx`, `transmit`, `PulseCode`); S3 has 4 TX + 4 RX channels. Not the older `TxChannel` trait style.
17 PCNT	Unstable. `Pcnt::new`, units 0-3 on S3, filter maximum 1023; exact channel setup methods not yet verified.
18 MCPWM	Unstable and partial (no dead-time, capture, fault). `McPwm::new(peripheral, PeripheralClockConfig)`; operator/timer calls come from the module example only.
19 TWAI / CAN	Unstable. `TwaiConfiguration::new(...)` signature read in 1.0.0-beta.1; must be re-verified for 1.1.1. Needs an external transceiver.
20 Wi-Fi, 21 BLE	Provided by the separate `esp-radio` crate plus an RTOS/executor crate (`esp-rtos`); versions matching esp-hal 1.1.1 not yet confirmed.
22 USB Serial/JTAG and OTG	Both partial/unstable; GPIO19/20 are reserved. Not yet verified.
23 Embassy	Needs `esp-rtos` + `embassy-executor` + `embassy-time` with versions that match `esp-metadata-generated 0.4`; not yet verified.

Part III - ESP-HAL 1.1.1 API reference (verified subset)

Stable means usable without the `unstable` Cargo feature. Unstable APIs are marked **U**. "Src" means the signature was read in 1.1.1; "1.1.0" or "1.0.0" means it was read in an older release and needs a compile check.

Initialisation and clocks

API	Module	Signature / behavior	Stab.	Src
`#[main]`	`esp_hal`	Marks `fn() -> !` as entry; needs feature `rt` (default).	Stable	1.1.1
`init`	`esp_hal`	`fn init(Config) -> Peripherals`; disables SWD/RWDT/TIMG watchdogs, sets clocks.	Stable	1.1.1
`Config::with_cpu_clock`	`esp_hal`	`(self, CpuClock) -> Self`. With `unstable` it accepts `impl Into<ClockConfig>`.	Stable	1.1.1
`CpuClock`	`esp_hal::clock`	`_80MHz`, `_160MHz`, `_240MHz`; `CpuClock::max()`.	Stable	1.1.1

GPIO

API	Module	Signature / behavior	Stab.	Src
`Output::new`	`esp_hal::gpio`	`(pin: impl OutputPin, Level, OutputConfig) -> Output`	Stable	1.1.1
`Output::set_high/set_low/toggle/set_level`	`gpio`	`&mut self`; `is_set_high()`, `is_set_low()` read the output latch	Stable	1.1.1
`Input::new`	`gpio`	`(pin: impl InputPin, InputConfig) -> Input`	Stable	1.1.1
`Input::is_high/is_low/level`	`gpio`	Read the pin level	Stable	1.1.1
`InputConfig::with_pull`	`gpio`	`Pull::{None, Up, Down}`	Stable	1.1.1
`OutputConfig::with_*`	`gpio`	`with_drive_mode`, `with_drive_strength`, `with_pull`	Stable	1.1.1
`Input::wait_for_*_edge` etc.	`gpio`	`async fn` waits; need an executor	Stable	1.1.1
`Io::new`, `set_interrupt_handler`	`gpio`	`Io::new(IO_MUX)`; installs the shared GPIO handler	U	1.1.1

API	Module	Signature / behavior	Stab.	Src
`Input::listen/unlisten/clear_interrupt/is_interrupt_set`	`gpio`	`listen(Event)`; status must be cleared manually	U	1.1.1
`#[handler]`	`esp_hal`	Attribute for interrupt handler functions; optional `priority`	U	1.1.1

Time and delay

API	Module	Signature / behavior	Stab.	Src
`Instant::now/elapsed`	`esp_hal::time`	1 microsecond counter; wraps after about 7 years	Stable	1.1.1
`Duration::from_*`	`time`	`from_micros/millis/secs/minutes/hours(u64)`	Stable	1.1.1
`Rate::from_hz/khz/mhz`	`time`	`(u32) -> Rate`	Stable	1.1.1
`Delay::new`, `delay_millis/micros/nanos`	`esp_hal::delay`	`&self` blocking delays	U (module)	1.1.1

Timers and watchdog

API	Module	Signature / behavior	Stab.	Src
`TimerGroup::new`	`esp_hal::timer::timg`	Fields `timer0`, `timer1`, `wdt`	U	1.1.1
`PeriodicTimer::new/start/wait/cancel`	`esp_hal::timer`	`start(Duration) -> Result<(), Error>`; `listen`, `set_interrupt_handler` available	U	1.1.1
`OneShotTimer::schedule/delay_millis`	`timer`	`schedule(Duration) -> Result`; async variants exist	U	1.1.1
`SystemTimer::new`	`timer::systimer`	Fields `alarm0..2`	U	1.1.1
`Wdt::enable/feed/set_timeout/disable`	`timer::timg`	`set_timeout(MwdtStage, Duration)`	U	1.1.1

UART

API	Module	Signature / behavior	S
`Uart::new`	`esp_hal::uart`	`(impl Instance, Config) -> Result<Uart<Blocking>, ConfigError>`	S
`with_rx / with_tx`	`uart`	`(self, impl PeripheralInput/Output) -> Self`	S
`write / flush / read / read_ready`	`uart`	`write(&[u8]) -> Result<usize, TxError>;`	S

API	Module	Signature / behavior	S
		`read(&mut [u8]) -> Result<usize, RxError>`	
`into_async`, `write_async`, `read_async`	`uart`	Async mode	S
`Config::with_baudrate/with_data_bits/with_parity/with_stop_bits`	`uart`	Default 115200 8N1; baud must be 1..5,000,000	S
`split`, `listen`, `read_buffered`, `UartTx/UartRx::new`	`uart`	Interrupts and split halves	U

I2C master

API	Module	Signature / behavior	Stab.	Src
`I2c::new`	`esp_hal::i2c::master`	`(impl Instance, Config) -> Result<I2c<Blocking>, ConfigError>`	Stable	1.1.1
`with_sda / with_scl`	`i2c::master`	Open-drain + internal pull-up	Stable	1.1.1
`write / read / write_read / transaction`	`i2c::master`	Address is `impl Into<I2cAddress>` (a `u8` works); zero-length read -> `ZeroLengthInvalid`	Stable	1.1.1
`Config::with_frequency`	`i2c::master`	`Rate`; default 100 kHz	Stable	1.1.1
`into_async`, `*_async`	`i2c::master`	Async variants	Stable	1.1.1
`Error`	`i2c::master`	`AcknowledgeCheckFailed(reason)`, `Timeout`, `ArbitrationLost`, `FifoExceeded`, …	Stable	1.1.1

SPI master

API	Module	Signature / behavior	Stab.	Src
`Spi::new`	`esp_hal::spi::master`	`(impl Instance, Config) -> Result<Spi<Blocking>, ConfigError>`	Stable	1.1.1
`with_sck / with_mosi / with_miso`	`spi::master`	Pin assignment	Stable	1.1.1
`read / write / transfer`	`spi::master`	Blocking, `&mut [u8]`; implements embedded-hal `SpiBus`	Stable	1.1.1
`Config::with_frequency / with_mode`	`spi::master`	`Rate`; `Mode::_0.._3`; 70 kHz-80 MHz	Stable	1.1.1
`with_cs`	`spi::master`	Only one hardware CS line	U	1.1.1
`with_dma`, `with_buffers`, half-duplex	`spi::master`	DMA and multi-line modes	U	1.1.1

ADC

API	Module	Signature / behavior	Stab.	Src
`AdcConfig::enable_pin`	`esp_hal::analog::adc`	`(&mut self, pin, Attenuation) -> AdcPin`	U	1.1.1
`Adc::new`	`analog::adc`	`(ADCx, AdcConfig) -> Adc<Blocking>`	U	1.1.1
`Adc::read_oneshot`	`analog::adc`	`(&mut AdcPin) -> nb::Result<u16, ()>`	U	1.1.1
`Attenuation`	`analog::adc`	`_0dB, _2p5dB, _6dB, _11dB`	U	1.1.1

LEDC

API	Module	Signature / behavior	Stab.	Src
`Ledc::new, set_global_slow_clock`	`esp_hal::ledc`	`LSGlobalClkSource::APBClk`	U	1.0.0 / 1.1.0
`Ledc::timer / channel`	`ledc`	`timer::<LowSpeed>(Number), channel::<LowSpeed>(Number, pin)`	U	1.0.0 / 1.1.0
`timer::config::Config`	`ledc::timer`	`{ duty, clock_source, frequency: Rate };` `Duty1Bit..Duty14Bit` on S3	U	1.1.0
`channel::config::Config`	`ledc::channel`	`{ timer, duty_pct, drive_mode }`	U	1.1.0
`ChannelIFace::set_duty`	`ledc::channel`	`(&self, pct: u8) -> Result`; also `start_duty_fade`	U	1.1.0

Part IV - Integrated projects

Project 1 - Basic monitoring system (ADC + PWM + LCD)

Status: partly checked against esp-hal 1.1.1, partly against 1.1.0 / 1.0.0 source (see notes). NOT compiled. Treat as an implementation pattern and let the compiler confirm names.

Architecture

```
  Potentiometer / LDR divider
            |  analog (0-3.3 V)
            v
   GPIO1 -> ADC1 -> raw 0..4095 --+--> percent --> LEDC channel0 -> GPIO2 -> LED
                                  |
                                  +--> LCD1602 (I2C0: SDA GPIO41, SCL GPIO40, 0x27)
                                  +--> serial console (esp-println)
```

Components and pin allocation

Signal	GPIO	Peripheral
Analog sensor	1	ADC1 channel 0
LED (PWM, 5 kHz, 8-bit)	2	LEDC timer0 / channel0
LCD SDA	41	I2C0
LCD SCL	40	I2C0

No camera pin (GPIO4-13, 15-18) is used. Components: board, 10 kohm potentiometer, LED, 330 ohm resistor, LCD1602 with I2C backpack (voltage note in lab 10).

Cargo dependencies

Use the Cargo.toml from section 1.5 unchanged.

Source code

File: `src/bin/project1_monitor.rs`, using the driver in `src/lcd1602.rs`.

```
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_println::println;

esp_bootloader_esp_idf::esp_app_desc!();

use core::fmt::Write;
use esp32s3_lab::lcd1602::Lcd;
use esp_hal::{
    analog::adc::{Adc, AdcConfig, Attenuation},
    delay::Delay,
    gpio::DriveMode,
    i2c::master::{Config as I2cConfig, I2c},
    ledc::{
        channel::{self, ChannelIFace},
        timer::{self, TimerIFace},
        LSGlobalClkSource, Ledc, LowSpeed,
    },
```

```
    main,
    time::Rate,
};

// Pins: ADC sensor GPIO1 | LED (PWM) GPIO2 | LCD SDA GPIO41, SCL GPIO40
#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default());

    let mut adc_cfg = AdcConfig::new();
    let mut sensor = adc_cfg.enable_pin(p.GPIO1, Attenuation::_11dB);
    let mut adc1 = Adc::new(p.ADC1, adc_cfg);

    let mut ledc = Ledc::new(p.LEDC);
    ledc.set_global_slow_clock(LSGlobalClkSource::APBClk);
    let mut t0 = ledc.timer::<LowSpeed>(timer::Number::Timer0);
    t0.configure(timer::config::Config {
        duty: timer::config::Duty::Duty8Bit,
        clock_source: timer::LSClockSource::APBClk,
        frequency: Rate::from_khz(5),
    })
    .unwrap();
    let mut led = ledc.channel::<LowSpeed>(channel::Number::Channel0, p.GPIO2);
    led.configure(channel::config::Config {
        timer: &t0,
        duty_pct: 0,
        drive_mode: DriveMode::PushPull,
    })
    .unwrap();

    let i2c = I2c::new(p.I2C0, I2cConfig::default())
        .unwrap()
        .with_sda(p.GPIO41)
        .with_scl(p.GPIO40);
    let mut lcd = Lcd::new(i2c, 0x27);
    lcd.init();

    let delay = Delay::new();
    loop {
        let raw: u16 = nb::block!(adc1.read_oneshot(&mut sensor)).unwrap();
        let pct = ((raw as u32 * 100) / 4095).min(100) as u8;
        led.set_duty(pct).unwrap();

        lcd.set_cursor(0, 0);
        write!(lcd, "ADC:{:4} {:3}% ", raw, pct).ok();
        lcd.set_cursor(0, 1);
        write!(lcd, "LED duty {:3}%  ", pct).ok();
        println!("raw={} pct={}", raw, pct);
        delay.delay_millis(200);
    }
}
```

Software architecture

A single super-loop: sample the ADC, convert to percent, update the LED duty, redraw both display rows with fixed-width formats (so old digits are overwritten), print to the console and wait 200 ms. All drivers are created in `main` and own their peripherals. Nothing is shared with interrupts, so no locking is required.

Build, flash and test

```
cargo run --release --bin project1_monitor
```

- Turn the potentiometer: ADC value and percentage change on the LCD and in the console.
- LED brightness follows the percentage; at 0 the LED is off.
- If the LCD stays blank, run lab 10 first to confirm the I2C address.

Project 2 - RFID access control (Stage 2 plan)

ESP32-S3 + RC522 (SPI) + LCD1602 + servo. Planned allocation, conflict-free with the camera: SPI2 SCK=GPIO47, MOSI=GPIO48, MISO=GPIO42, RC522 CS=GPIO38 (plain GPIO output, because `with_cs` is unstable), RC522 RST=GPIO39, servo=GPIO14 (LEDC), LCD on GPIO41/40. The RC522 3.3 V supply is mandatory. Source code is not written yet.

Project 3 - IoT monitor with Embassy (Stage 2 plan)

Sensors + LCD + Wi-Fi with async tasks. Depends on verifying the esp-rtos / esp-radio / embassy versions that match esp-hal 1.1.1. Use ADC1 pins (GPIO1-3) for analog sensors because ADC2 conflicts with Wi-Fi.

Appendix A - Project directory

```
esp32s3-lab/
|-- Cargo.toml
|-- rust-toolchain.toml
|-- build.rs
|-- .cargo/config.toml
|-- src/
|   |-- lib.rs                 (shared modules)
|   |-- lcd1602.rs             (PCF8574 LCD driver)
|   `-- bin/
|       |-- lab01_gpio_output.rs   lab02_gpio_input.rs   lab03_cpu_clock.rs
|       |-- lab04_adc.rs           lab05_ledc_pwm.rs     lab06_servo.rs
|       |-- lab07_timer.rs         lab08_gpio_interrupt.rs
|       |-- lab09_uart.rs          lab10_i2c_scan.rs     lab11_lcd1602.rs
|       |-- lab12_spi_loopback.rs  project1_monitor.rs
```

Appendix B - Checklist per laboratory

#	Step	Done
1	`espup` toolchain loaded (`. ~/export-esp.sh`), `rustc +esp --version` works	
2	Board connected, user in `dialout`, `espflash board-info` detects the chip	
3	Wiring checked against the lab table; camera pins untouched; no 5 V on any GPIO	
4	`cargo build --release --bin <lab>` compiles; fix names the compiler reports	
5	`cargo run --release --bin <lab>` flashes and opens the monitor	
6	Observed behavior matches "Expected behavior"; record the result	
7	If different, read the lab troubleshooting list before editing code	

Appendix C - Sources

- esp-hal 1.1.1 ESP32-S3 API: https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- esp-hal 1.1.1 source on docs.rs: https://docs.rs/crate/esp-hal/1.1.1/source/
- esp-hal repository: https://github.com/esp-rs/esp-hal
- The Rust on ESP Book: https://esp-rs.github.io/book/
- The Embedded Rust Book: https://docs.rust-embedded.org/book/
- Embassy: https://embassy.dev/